AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

Postgraduate Programs
Spring 2026

CSE323: Advanced Embedded Systems Design
CSE411/CSE411s: Real-Time and Embedded Systems Design

Assignment-07

Objective

The goal of this assignment is to demonstrate the capability of Counting Semaphores to queue and manage multiple consecutive hardware interrupts that fire during a task's execution, ensuring no interrupt events are lost.

Description

In a real-time system, a busy task might be interrupted multiple times before it can process the first event. While Binary Semaphores drop these subsequent events, Counting Semaphores successfully tally them.

The standard course material simulates this by triggering a single interrupt and issuing multiple xSemaphoreGiveFromISR calls within that single ISR. **You are not allowed to use this simulation method.** Instead, you must design a system where *actual, distinct hardware interrupts* are fired sequentially while the FreeRTOS handler task is occupied.

Task Requirements

- **The Handler Task:** Create a task that waits on the counting semaphore. When it unblocks, it should increment a global TokensProcessed variable, print/log the event, and then enter a long, blocking delay (e.g., vTaskDelay(2000)) to simulate heavy processing.
- **The Trigger Task:** Create a separate, lower-priority task that forcefully triggers a specific hardware interrupt **3 times in a row** without waiting.
    - You can forcefully trigger a hardware interrupt through software by writing to the NVIC Interrupt Set-Pending Register, or by using CMSIS functions like NVIC_SetPendingIRQ(IRQn).
- **The ISR:** Inside the ISR, give the counting semaphore (xSemaphoreGiveFromISR) and clear the pending flag.

What your report must demonstrate

- A clear, technical explanation of how you used the NVIC pending registers to forcefully trigger the consecutive hardware interrupts.
- **Proof of Execution:** You must run the Keil Simulator in debug mode, add your `TokensProcessed` variable to the **Keil Logic Analyzer**, and take a screenshot. The resulting graph must show a "staircase" effect—a flat line that suddenly steps up rapidly by 3, proving the task processed the queued tokens sequentially once it unblocked.

Submission format

Submit ONE PDF document containing your methodology, explanation of the pending register manipulation, and the required execution evidence.